

SIH - Identify Site Gaps

Ok, so point out and summarise what are the things our project should have which nadres already have , and what are the things that we need to add

Yes. After comparing **your PashuShield PRD + current prototype structure** with the actual NADRES V2 capabilities, the cleanest approach is to divide the features into “**already proven by NADRES**” and “**our actual differentiators**.”

Your PRD itself is designed around the workflow **Report → Detect → Assess → Refer → Treat → Prevent → Monitor**.

1. Things PashuShield should have because NADRES already does them

These are **not your main innovation**. You should include them because they are important to the problem, but don't claim them as unique features.

Feature	NADRES	PashuShield should have
Disease forecasting	Yes — AI/ML forecasting, up to 2 months	Yes
Disease risk assessment	Yes	Yes
Historical disease trends	Yes	Yes
Weather/environment data	Yes — weather, remote sensing, etc.	Yes
GIS / risk maps	Yes	Yes
District-level risk	Yes	Yes
Disease alerts	Yes	Yes
SMS alerts	Yes, including farmer alerts through FRUITS	Yes

Feature	NADRES	PashuShield should have
Disease surveillance data	Yes	Yes
AI/ML models	Yes	Yes
Government/official monitoring	Yes	Yes
Disease reporting	Yes, including a Disease Report Survey	Yes, but make it much more complete

NADRES currently provides AI/ML disease forecasting, weather/environmental integration, GIS/risk maps, disease alerts and SMS dissemination. 

So your basic PashuShield foundation should be:

AI + Disease Data + Weather + GIS + Forecasting + Alerts + Government Dashboard

These are the things NADRES already demonstrates well.

2. Things PashuShield needs to ADD

This is where your project should differentiate itself.

Your PRD already identifies these gaps and builds the platform around them.

A. Real-time farmer reporting

Instead of only asking whether disease symptoms were observed, PashuShield should allow:

Farmer → Report → Symptoms + mortality + location + animal/herd details + photo/video

Your PRD specifies reporting within **60 seconds**, including animal type, age, breed, GPS, symptoms, affected count, mortality, media, vaccination status, treatment and onset date.

This should be one of your strongest differentiators.

B. AI-assisted triage

NADRES is mainly focused on **forecasting disease risk**.

PashuShield should add:

"This particular reported case looks high-risk."

Input:

Symptoms + animal type + herd exposure + nearby cases

Output:

Risk = 82/100

Suspected category = contagious disease

Action = isolate + contact vet + collect sample

Your PRD explicitly defines this risk-score and suspected-category workflow.

C. Individual/herd health records

This is a major addition.

Maintain:

- Herd ID
- Animal information
- Vaccination history
- Treatment history
- Previous illnesses
- Current health status

Your MVP requires a **herd-level health record**, with individual animal records planned as an extension.

D. Veterinary case management

Instead of simply showing a warning, PashuShield should create a **case**.

Farmer report



```

AI triage
  ↓
High-risk case
  ↓
Veterinarian dashboard
  ↓
Assign field worker

```

The veterinarian should be able to review the case, assign a worker, request a sample, recommend treatment, escalate or close it.

E. Laboratory referral + sample tracking

This is one of the **best differentiators**.

NADRES is strong on disease forecasting, but your system should close the loop:

```

Suspected case
  ↓
Sample collection
  ↓
Sample ID
  ↓
Laboratory
  ↓
Testing
  ↓
Result
  ↓
Case updated

```

Your PRD specifically requires sample chain-of-custody and status such as **In Transit** → **Received** → **Result Pending** → **Resulted**.

F. Automatic outbreak cluster detection

Don't only display NADRES-style risk maps.

PashuShield should analyse **live reports**:

4 farms → same symptoms → nearby villages → increasing cases

Then:

Emerging Outbreak Detected

with:

- Case growth rate
- Number of affected villages
- Geographic radius
- Transmission risk

This is explicitly part of your MVP.

G. Vaccination management

Not just showing vaccination information.

PashuShield should calculate:

Village A: 42% coverage → HIGH PRIORITY

Then:

Vaccination campaign recommended

And maintain vaccination history/reminders.

Your PRD includes vaccination coverage, low-coverage prioritization and vaccination reminders.

H. Field-worker workflow

Add a dedicated field-worker mode:

Today's tasks

- Visit Case #104
- Collect sample
- Verify symptoms
- Vaccination follow-up
- GPS verification

This creates the missing connection between **digital report and physical action**.

I. Offline-first reporting

This is particularly important for the problem statement.

Farmer should be able to:

Report → Save locally → Internet unavailable → Automatically sync later

Your PRD explicitly requires offline capture and synchronization, with a pilot target of >95% successful sync.

J. Low-connectivity + multilingual access

Your system should work through:

- Lightweight mobile/web
- Regional languages
- Low-bandwidth media
- Eventually IVR

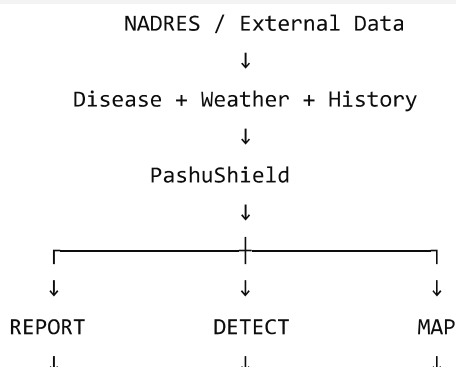
IVR is in your **Phase 2**, not MVP, so don't claim it as implemented in the prototype.

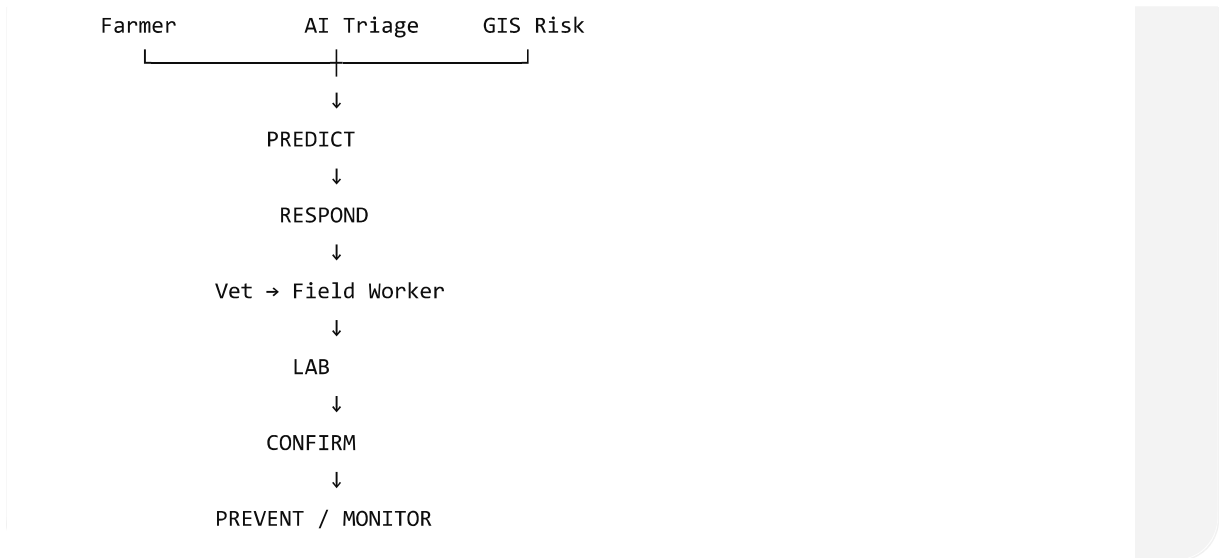
3. One important thing: don't duplicate NADRES unnecessarily

You don't need to build a completely new forecasting engine just to say you have forecasting.

NADRES already has sophisticated forecasting using disease history and environmental variables, including RF, ANN and GLM-based approaches. 

Instead, your architecture should ideally be:





Your current prototype already has separate **Report, Vet, Lab and Command Center** pages, plus a Leaflet-based map component, according to the uploaded project structure.

4. Final division for your presentation

Already available in NADRES — use as foundation

- AI/ML disease forecasting
- Disease risk prediction
- Historical disease data
- Weather/environmental data
- GIS/risk maps
- District-level forecasting
- Disease surveillance
- Disease alerts
- SMS dissemination
- Government/official monitoring

PashuShield's added value

- Real-time farmer/field-worker reporting
- AI symptom-based triage
- Individual/herd health records
- Vaccination & treatment history
- Live outbreak cluster detection
- Veterinary case management
- Field-worker task assignment
- Laboratory referral

- Sample tracking
- Lab-result → case update
- Automated action recommendations
- Offline-first reporting
- Low-connectivity support
- Role-based workflow
- End-to-end case tracking

The simplest way to explain the difference

NADRES mainly tells us: "Where and when is disease risk likely to increase?"

PashuShield adds: "A farmer has reported a case. What is the risk, who should respond, where should the sample go, what action should be taken, and has the case been resolved?"

That should be the **core positioning of PashuShield**. It makes your project an **end-to-end response and surveillance layer**, rather than trying to compete with NADRES purely on disease forecasting.



Livestock Health Surveill...
Document

based on this research Create a detail plan for the project

PashuShield — Detailed Project Implementation Plan

Based on the **Livestock Health Surveillance System** research, the PashuShield PRD, and the existing prototype scope, the project should be built as an **end-to-end livestock health surveillance and response system**, not just as an AI disease-detection application.

The central idea is:

Report → Detect → Assess → Refer → Treat → Prevent → Monitor

The research shows that existing systems are strong in individual areas—NADRES in macro forecasting, EMA-i in field reporting, Bharat Pashudhan in animal identity, AfyaData in

offline reporting, SaTScan in cluster detection, and LIMS in laboratory workflows—but the major opportunity is **connecting these capabilities into one continuous workflow.**

1. What We Should Actually Build

Core objective

Build a platform where:

Farmer reports symptoms

↓

AI/rule engine evaluates risk

↓

Case appears on GIS map

↓

System detects nearby clusters

↓

Veterinarian receives priority case

↓

Field worker is assigned

↓

Sample is collected and tracked

↓

Lab result updates the case

↓

Vaccination/treatment/prevention action is recorded

↓

Government dashboard sees the overall situation

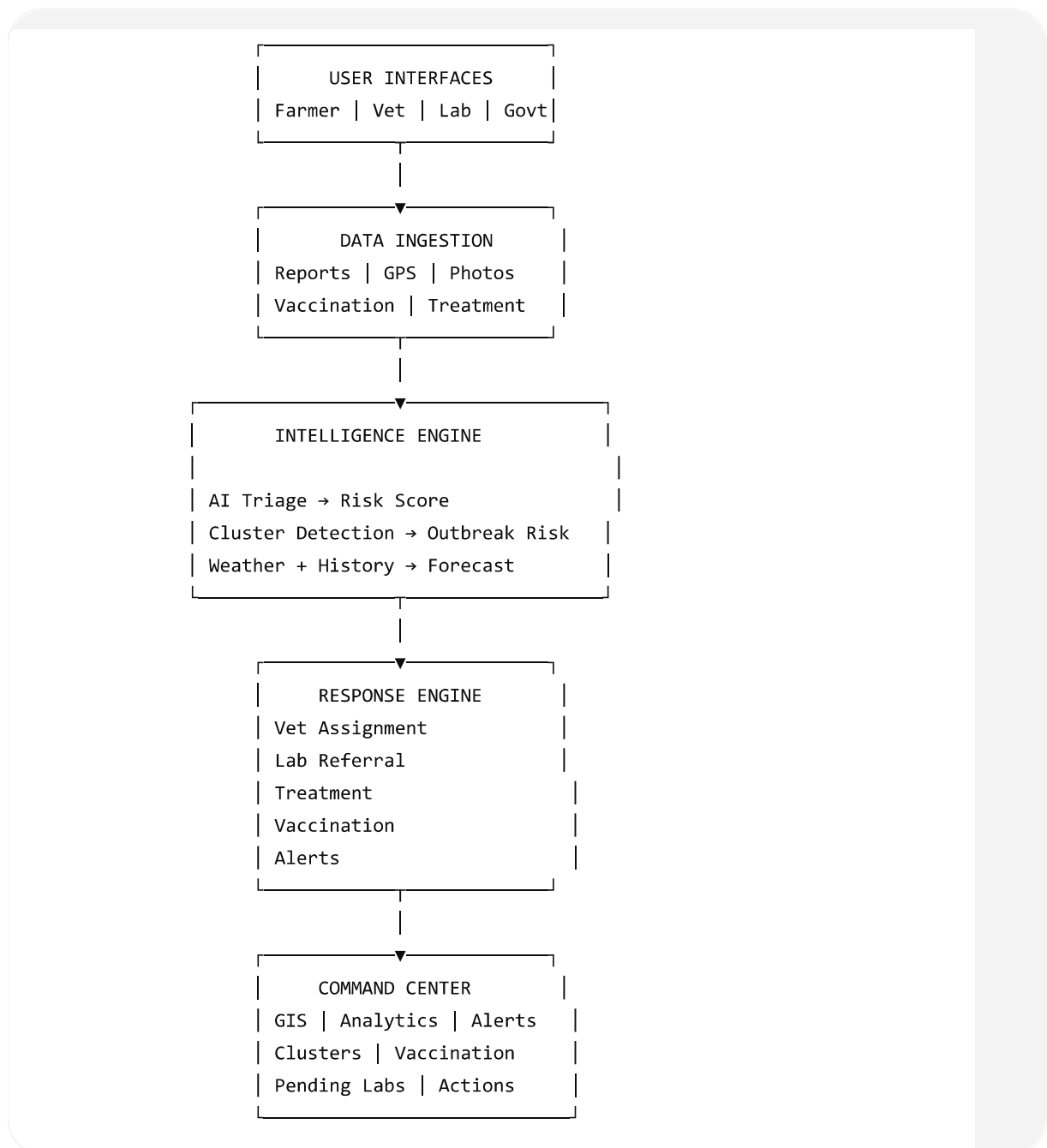
↓

Confirmed data improves future prediction

This directly follows the architecture proposed in the research and the PRD.

2. Project Architecture

Use a 5-layer architecture.



3. Module 1 — Farmer / Field Reporting

Goal

Make disease reporting possible in **less than 60 seconds**.

The PRD specifically requires collection of animal type, age, breed, GPS, symptoms, number affected, mortality, media, vaccination status, treatment and onset date.

MVP interface

Create a simple form:

Step 1 — Animal

- Cow
- Buffalo
- Goat
- Sheep
- Pig
- Poultry

Step 2 — Symptoms

Use selectable buttons rather than requiring typing.

Example:

- Fever
- Cough
- Nasal discharge
- Mouth lesions
- Skin nodules
- Diarrhea
- Weakness
- Reduced milk production
- Sudden death
- Swelling

Step 3 — Severity

Animals affected: 3
Deaths: 1

Symptoms started: 2 days ago

Step 4 — Evidence

- Camera/photo
- Optional video
- GPS location

Step 5 — Vaccination

Vaccinated?

YES / NO / UNKNOWN

Step 6

SUBMIT REPORT

4. Offline-First Reporting

This should be one of PashuShield's strongest differentiators.

The research specifically identifies poor connectivity as a major failure point in existing systems and highlights AfyaData's offline-first architecture as a useful model.

Workflow

```
No Internet
  ↓
User fills report
  ↓
Save locally
  ↓
Generate Report ID
  ↓
"Pending Sync"
  ↓
Internet available
  ↓
Automatic synchronization
  ↓
Server
```

Important implementation

Every locally created report should have:

```
client_report_id
created_at
device_id
sync_status
```

When synchronization happens, the backend checks the `client_report_id` so the same report is not uploaded twice.

This follows the PRD's requirement for unique client-generated IDs and idempotent synchronization.

5. Module 2 — AI Disease Triage

This is where PashuShield should use AI.

Important:

Do NOT present this as disease diagnosis.

The system should say:

Suspected contagious disease — High Risk

and:

Veterinary confirmation required

The PRD explicitly requires this guardrail.

MVP AI approach

Do **not** attempt to build a complex medical AI model from scratch for the hackathon.

Use a **hybrid system**:

```
Symptoms
+
Animal type
+
Number affected
+
```

```
Mortality
+
Vaccination status
+
Nearby cases
↓
Rule Engine
+
Lightweight ML Model
↓
Risk Score 0-100
↓
Suspected Disease Category
↓
Recommended Action
```

Example

Input:

```
Animal: Cow
Symptoms:
- Fever
- Mouth lesions
- Excessive salivation

Affected: 3
Deaths: 0
Vaccination: Unknown
Nearby cases: 4
```

Output:

```
Risk Score: 87/100

Risk Level:
HIGH

Suspected Category:
Contagious / Vesicular Disease

Possible Concern:
FMD-like syndrome

Actions:
✓ Isolate affected animals
✓ Contact veterinarian
✓ Avoid animal movement
```

- ✓ Consider sample collection

Veterinary confirmation required

6. AI Model Strategy

For the prototype, use:

Primary

Rule-based scoring

Example:

Fever	+15
Mouth lesions	+25
Multiple animals	+20
Recent mortality	+20
Nearby similar cases	+15
Low vaccination	+10

	105

Cap at 100.

Then convert:

Score	Risk
0–25	Normal
26–50	Watch
51–75	Suspected
76–100	High Risk

This is more defensible for an MVP than claiming that a trained medical AI model has clinical accuracy.

Later

Replace/augment the rule engine with:

- Bayesian Network

- Random Forest
- XGBoost
- Neural Network

The research specifically proposes Bayesian reasoning for differential clinical support.

7. Module 3 — GIS Disease Map

This is already an important part of your prototype architecture.

The system should plot every report geographically.

Map layers

```
Reported Cases
+
Cluster Areas
+
Vaccination Coverage
+
Risk Forecast
+
Veterinary/Lab Resources
```

Risk colors

```
NORMAL
WATCH
SUSPECTED CLUSTER
OUTBREAK RISK
```

These categories align with the PRD's GIS requirements.

8. Module 4 — Outbreak / Cluster Detection

This is where PashuShield becomes more than a reporting application.

Suppose:

```
Village A → 3 cases
Village B → 4 cases
Village C → 5 cases
```

within:

10 km

during:

48 hours

The system should identify:

Emerging Disease Cluster

MVP algorithm

Start with a simple spatial-temporal algorithm.

Input

latitude
longitude
timestamp
symptom category
case count

Process

```
New report
  ↓
Find cases within X km
  ↓
Check last X days
  ↓
Count similar cases
  ↓
Calculate growth rate
  ↓
Check vaccination coverage
  ↓
Calculate cluster score
```

Example

```
Cases in previous 7 days = 4
Cases in current 48 hours = 12

Growth = +200%

Affected villages = 3

Radius = 8 km

Vaccination coverage = 42%
```

Output:

```
EMERGING OUTBREAK

Transmission Risk: HIGH
Affected Villages: 3
Radius: 8 km
Growth: +200%
Vaccination Coverage: 42%
```

The PRD specifically calls for combining growth rate, geographic clustering and vaccination coverage.

9. Advanced Cluster Detection — Phase 2

Once the basic algorithm works, integrate the methodology from **SaTScan**.

The research identifies SaTScan as particularly useful for detecting localized spatial-temporal anomalies that macro-level forecasting may miss.

Architecture

```
Daily Reports
  ↓
Spatial-temporal engine
  ↓
Scan geographic windows
  ↓
Calculate expected vs observed cases
  ↓
Relative Risk
  ↓
Cluster significance
```

↓
Alert

Do **not** make SaTScan a dependency for the hackathon MVP unless your team already has time to integrate it.

10. Module 5 — Historical + Predictive Risk

This is where NADRES becomes your reference/foundation.

The research describes NADRES v2 as a macro-level system using weather, remote sensing, historical disease and livestock-density information for disease forecasting.

PashuShield should complement this rather than pretend to replace it.

Data

```
Historical cases
+
Current cases
+
Animal density
+
Temperature
+
Humidity
+
Rainfall
+
Season
↓
Risk Model
↓
Future Risk
```

11. MVP Forecasting

For the hackathon, keep it simple.

Show:

Historical

Week 1 → 12 cases
Week 2 → 15
Week 3 → 18
Week 4 → 27

Then:

Next week projected:
32 cases

Label it clearly:

Risk forecast — not a guaranteed prediction

This is consistent with the PRD.

12. Phase 2 Weather Intelligence

Later integrate:

- Temperature
- Humidity
- Rainfall
- Flood conditions
- Season
- NDVI
- LST

The research specifically describes NADRES using environmental and remote-sensing variables such as LST, NDVI and precipitation.

Then:

Weather
+
Disease history
+
Current cases
+
Animal density
↓
ML model
↓
30-60 day disease-risk forecast

13. Module 6 — Herd Health Record

Each farmer should have a **Herd Profile**.

Example:

Herd ID: H-10482

Species:

Cattle

Total animals:

27

Vaccinated:

19 / 27

Previous diseases:

LSD – 2025

Recent treatment:

Antibiotic – Aug 2026

Current suspected cases:

3

For the MVP, keep it **herd-level**.

The PRD explicitly places individual animal passports in a later phase.

14. Animal Identity — Future

For the full product:

Animal Tag ID

↓

Animal Profile

↓

Vaccination

↓

Disease

↓

Treatment

↓

Lab Results

↓

Complete Health History

The research takes inspiration from Bharat Pashudhan's unique animal identity approach, but also highlights the operational problems caused by excessive online verification and connectivity requirements.

So PashuShield should prioritize:

Low-friction identity + offline capability

rather than copying the administrative friction.

15. Module 7 — Veterinary Dashboard

The veterinarian should not see every case with equal priority.

Dashboard

HIGH RISK CASES	12
EMERGING CLUSTERS	3
PENDING SAMPLES	7
FIELD VISITS	15

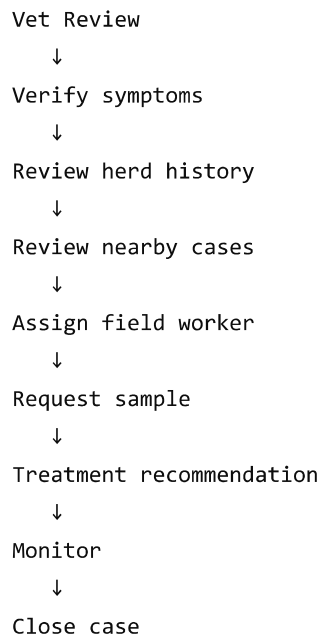
Case queue

Case	Risk	Location	Action
#1024	High	Village A	Investigate
#1025	High	Village A	Collect sample
#1026	Medium	Village B	Review
#1027	Low	Village C	Monitor

16. Vet Case Workflow

New Case

↓



```
graph TD; A[Vet Review] --> B[Verify symptoms]; B --> C[Review herd history]; C --> D[Review nearby cases]; D --> E[Assign field worker]; E --> F[Request sample]; F --> G[Treatment recommendation]; G --> H[Monitor]; H --> I[Close case];
```

Vet Review
↓
Verify symptoms
↓
Review herd history
↓
Review nearby cases
↓
Assign field worker
↓
Request sample
↓
Treatment recommendation
↓
Monitor
↓
Close case

This is directly aligned with the PRD's veterinary case-management requirements.

17. Module 8 — Field Worker App

The field worker receives:

TODAY'S TASKS

1. Visit Village A
High-risk FMD-like case
2. Collect Sample
Case #1042
3. Verify vaccination
Village B
4. Follow-up
Case #998

GPS should be captured during:

- Case verification
- Farm visit
- Sample collection
- Vaccination

The PRD explicitly specifies these field-worker functions.

18. Module 9 — Laboratory Workflow

This is an important differentiator.

Sample lifecycle

```
graph TD; A[Sample Requested] --> B[Sample Collected]; B --> C[QR / Sample ID generated]; C --> D[In Transit]; D --> E[Lab Received]; E --> F[Testing]; F --> G[Result Pending]; G --> H[Resulted];
```

Example:

```
Sample ID: SMP-2026-00192

Case ID: CASE-1042
Animal/Herd: H-10482
Collected: 04 Sept
Collector: Field Worker #17
Destination: District Veterinary Lab

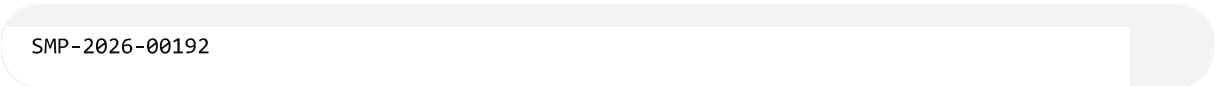
Status:
RESULT PENDING
```

The research specifically identifies the lack of connected sample traceability as a major architectural gap.

19. QR Code Integration

For the prototype:

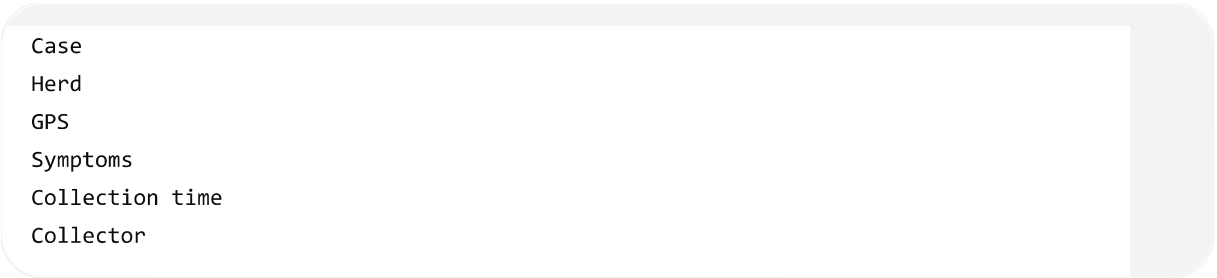
Generate QR



SMP-2026-00192

Scan it in the laboratory dashboard.

The system automatically loads:



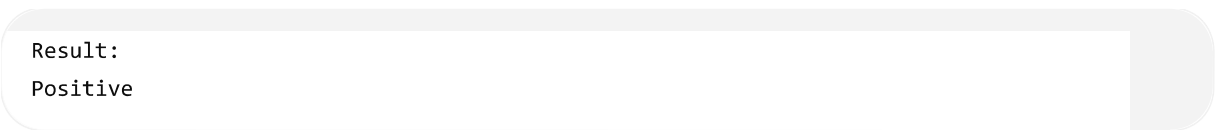
Case
Herd
GPS
Symptoms
Collection time
Collector

This demonstrates the **chain of custody** without needing actual RFID hardware.

20. Laboratory Result → Case Update

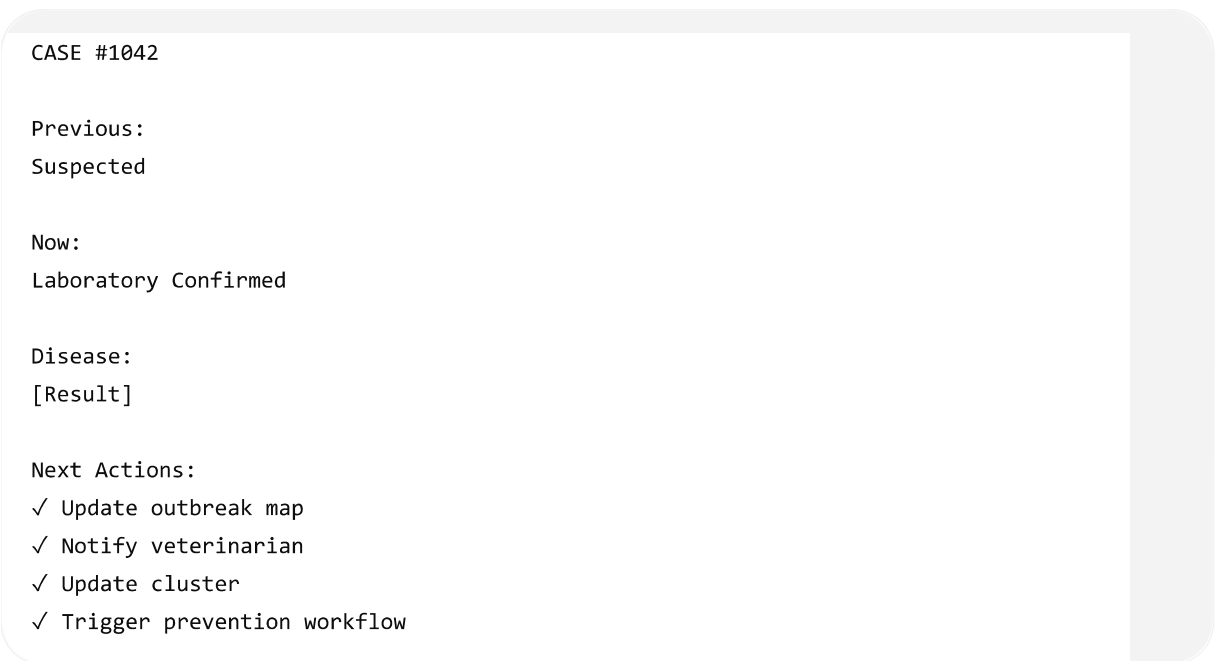
This is critical.

When lab technician enters:



Result:
Positive

the system automatically updates:



CASE #1042

Previous:
Suspected

Now:
Laboratory Confirmed

Disease:
[Result]

Next Actions:

- ✓ Update outbreak map
- ✓ Notify veterinarian
- ✓ Update cluster
- ✓ Trigger prevention workflow

This closes the loop identified in the research as missing from siloed laboratory systems.

21. Module 10 — Vaccination Management

Dashboard should show:

Village A
Vaccination Coverage: 42%
Risk: HIGH
Priority: URGENT

Calculate

Vaccinated animals
----- × 100
Total animals

Then identify:

Low vaccination
+
High disease risk
↓
Priority vaccination zone

The PRD specifically requires village/block/district vaccination coverage and low-coverage prioritization.

22. Module 11 — Alert Engine

Don't send the same message to everyone.

Farmer

"Several suspected disease cases have been reported nearby. Keep affected animals separate and contact your veterinarian."

Field Worker

"Visit required: 3 suspected cases in Village A."

Veterinarian

"High-risk cluster detected: 12 cases across 3 villages."

Government Official

"Priority intervention: Cluster expanding toward Village C. Vaccination coverage 42%."

This role-based alert structure is specified in the PRD.

23. Avoid Alert Fatigue

This is an important research-derived feature.

Don't trigger an outbreak alert from **one symptom report**.

Use multiple signals:

```
Symptom Risk
+
Case Growth
+
Spatial Clustering
+
Vaccination Coverage
+
Historical Baseline
↓
Final Alert Score
```

For example:

```
AI Risk > 70
AND
Cluster detected
AND
Case growth > threshold
```

→ **High-confidence alert**

The research specifically recommends combining clinical and spatial signals to reduce false-positive alerts.

24. Module 12 — Government Command Center

This should be the **main presentation screen**.

Top KPIs

ACTIVE CASES	128
HIGH-RISK CASES	21
CLUSTERS	5
PENDING LABS	17
VACCINATION	68%

Main map

Show:

- Disease cases
- Risk zones
- Emerging clusters
- Villages
- District boundaries

Priority actions

1. Investigate Village A cluster
2. Vaccinate Village B
3. Expedite 4 lab samples
4. Deploy field team to Zone C
5. Issue farmer advisory

This directly corresponds to the command-center requirements.

25. Recommended Database

Use PostgreSQL.

For geographic data, use **PostGIS**.

Main tables

```
users
farmers
field_workers
veterinarians
labs
```


herds
animals

health_reports
symptoms
treatments
vaccinations

cases
risk_scores
clusters

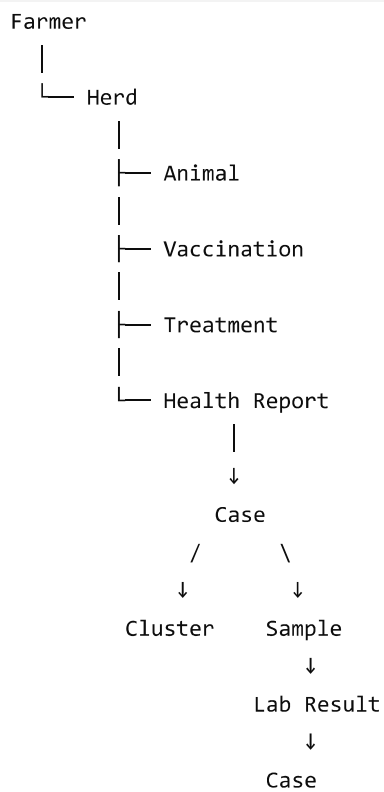
samples
lab_results

alerts
advisories

weather_data
disease_history

field_tasks
audit_logs

26. Key Relationships



↓
Dashboard

27. API Architecture

Use REST APIs for the MVP.

Reporting

```
POST /api/reports
GET /api/reports/:id
```

Triage

```
POST /api/triage
GET /api/triage/:caseId
```

Cases

```
GET /api/cases
GET /api/cases/:id
PATCH /api/cases/:id
```

Map

```
GET /api/map/cases
GET /api/map/clusters
GET /api/map/risk-zones
```

Lab

```
POST /api/samples
PATCH /api/samples/:id
POST /api/lab-results
```

Vaccination

```
POST /api/vaccinations
GET /api/vaccination/coverage
```

Alerts

```
GET /api/alerts
POST /api/alerts
```

28. Suggested Tech Stack

Since your existing prototype already uses **Next.js/React/TypeScript** and **Leaflet**, keep that rather than rebuilding the frontend.

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- Leaflet / React-Leaflet
- PWA

Backend

For the MVP:

- Next.js API routes **or**
- FastAPI

I would recommend:

Next.js + FastAPI

because the AI/data-processing layer can then be cleanly separated.

Database

- PostgreSQL
- PostGIS

AI/ML

MVP

- Python
- scikit-learn
- Rule engine
- Random Forest

Future

- Bayesian Network
- XGBoost
- Neural Networks
- Time-series forecasting

Maps

- Leaflet
- OpenStreetMap
- PostGIS

Offline

- PWA
- IndexedDB
- Background sync

Authentication

- JWT
- Role-based access control

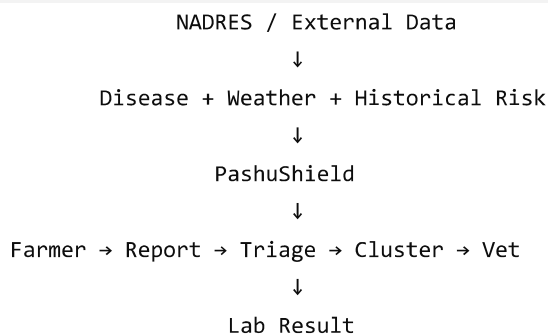
Notifications

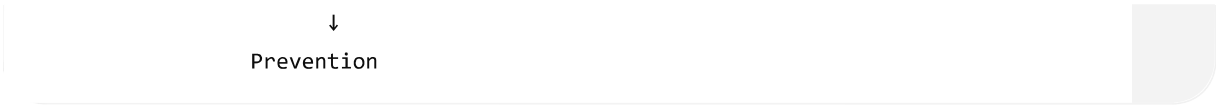
- SMS API
- WhatsApp API
- Push notifications

29. Where NADRES Fits

Do **not** position PashuShield as "another NADRES."

Instead:





The research identifies NADRES as particularly strong in macro-level weather-based forecasting, while the missing piece is granular bottom-up syndromic reporting.

Your positioning:

NADRES predicts where disease risk may increase. PashuShield captures what is actually happening on the ground and coordinates the response.

That is a much stronger project story.

30. Where Each Existing System Inspires PashuShield

Existing system	Take from it	PashuShield implementation
NADRES	Macro forecasting	Weather + historical risk
EMA-i	Field reporting	Farmer/field reporting
Bharat Pashudhan	Animal identity	Herd/animal records
AfyaData	Offline-first	Offline report + sync
SaTScan	Cluster detection	Spatiotemporal clusters
Vet LIMS	Sample workflow	Sample → lab → result
PashuShield	Integration	One continuous workflow

This is the central finding of your research.

31. Development Priority

Don't build everything simultaneously.

Priority 1 — Must Work

Report



Form → Database

Detect



Report → Risk Score

Map



Report → GPS → Map

Cluster



Reports → Cluster

Respond



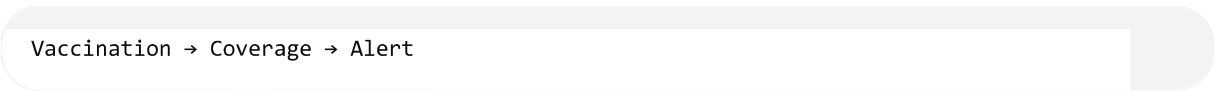
High Risk → Vet Dashboard

Lab



Case → Sample → Result

Prevent



Vaccination → Coverage → Alert

These are the six core MVP modules defined in the prototype scope.

32. What NOT to Build First

For the hackathon, avoid spending time on:

- Full Bayesian clinical model
- Training image-diagnosis model
- RFID hardware
- IoT temperature sensors
- Blockchain
- Full IVR

- National-scale deployment
- Complex satellite-data pipeline
- Individual animal passport
- Full WhatsApp NLP
- Advanced R_0 modelling

These are valuable future capabilities, but the PRD already identifies several of them as Phase 2/3 rather than MVP.

33. Development Roadmap

Phase 0 — Foundation

Day 1

- Finalize diseases
 - Define symptoms
 - Define risk levels
 - Database schema
 - User roles
 - UI flow
 - API structure
-

Phase 1 — Reporting

Day 2–3

Build:

- Farmer report page
- Symptom selection
- GPS
- Photo
- Herd information
- Offline storage
- Backend API

Output

Farmer → Report → Database

Phase 2 — AI Triage

Day 3–4

Build:

- Rule engine
- Risk score
- Suspected category
- Recommended actions
- Vet confirmation warning

Output

Report → AI → Risk Score

Phase 3 — GIS

Day 4–5

Build:

- Leaflet map
- Case markers
- Risk zones
- Cluster circles/polygons
- Village/block filtering

Output

Reports → GIS

Phase 4 — Cluster Detection

Day 5–6

Build:

- Radius search

- Time-window analysis
- Case-growth calculation
- Cluster score
- Outbreak alert

Output

Cases

↓

Spatial + Temporal Analysis

↓

Emerging Cluster

Phase 5 — Vet + Response

Day 6–7

Build:

- Vet dashboard
- Risk-based case queue
- Assign field worker
- Case status
- Recommended action
- Field visit

Phase 6 — Laboratory

Day 7–8

Build:

- Sample creation
 - QR code
 - Sample tracking
 - Lab dashboard
 - Result entry
 - Automatic case update
-

Phase 7 — Prevention

Day 8

Build:

- Vaccination records
 - Coverage calculation
 - Low-coverage detection
 - Vaccination priority areas
 - Alerts
-

Phase 8 — Command Center

Day 9

Build:

- District overview
 - Case statistics
 - Cluster map
 - Vaccination statistics
 - Pending samples
 - Priority actions
-

Phase 9 — Demo Hardening

Day 10

Test the **single complete story**, not individual pages.

34. The Demo Scenario

Use one realistic scenario throughout the presentation.

Step 1

Farmer opens PashuShield.

3 Cows
Fever

Mouth lesions
Excessive salivation

Uploads photo.

Step 2

AI responds:

Risk Score: 87/100

HIGH RISK

Suspected contagious/vesicular disease

Veterinary confirmation required

Step 3

Map automatically shows:

Farmer's location

+

4 nearby similar reports

Step 4

Cluster engine detects:

12 cases

3 villages

8 km radius

48 hours

and generates:

EMERGING OUTBREAK CLUSTER

Step 5

Vet dashboard receives:

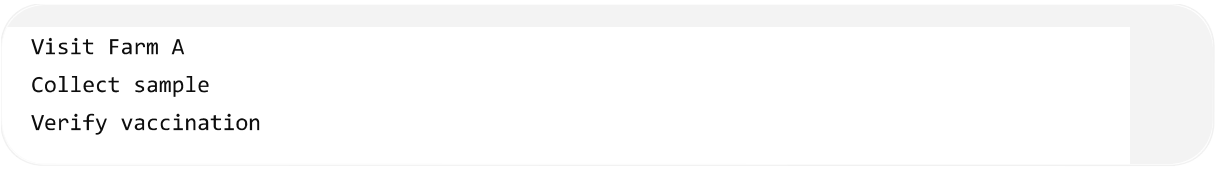


HIGH PRIORITY CASE

Vet assigns field worker.

Step 6

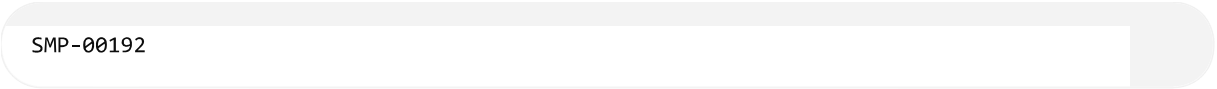
Field worker receives:



Visit Farm A
Collect sample
Verify vaccination

Step 7

Field worker creates:

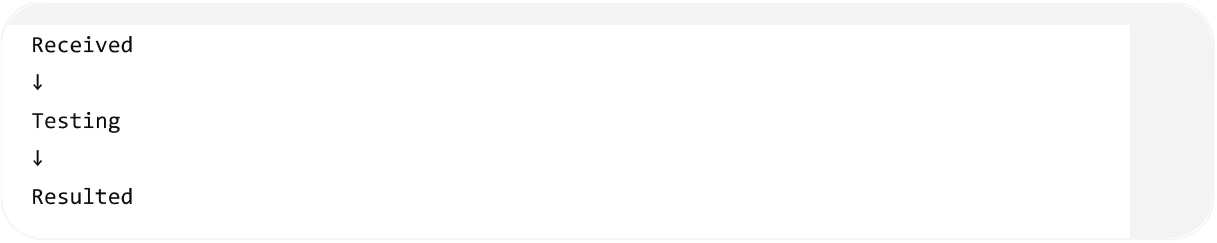


SMP-00192

and QR code is generated.

Step 8

Lab scans the sample.



Received
↓
Testing
↓
Resulted

Step 9

Lab enters result.

Positive

Step 10

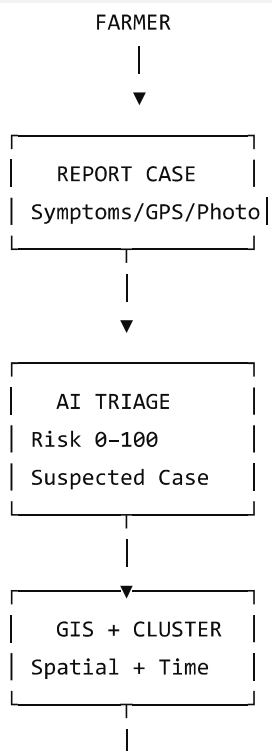
PashuShield automatically updates:

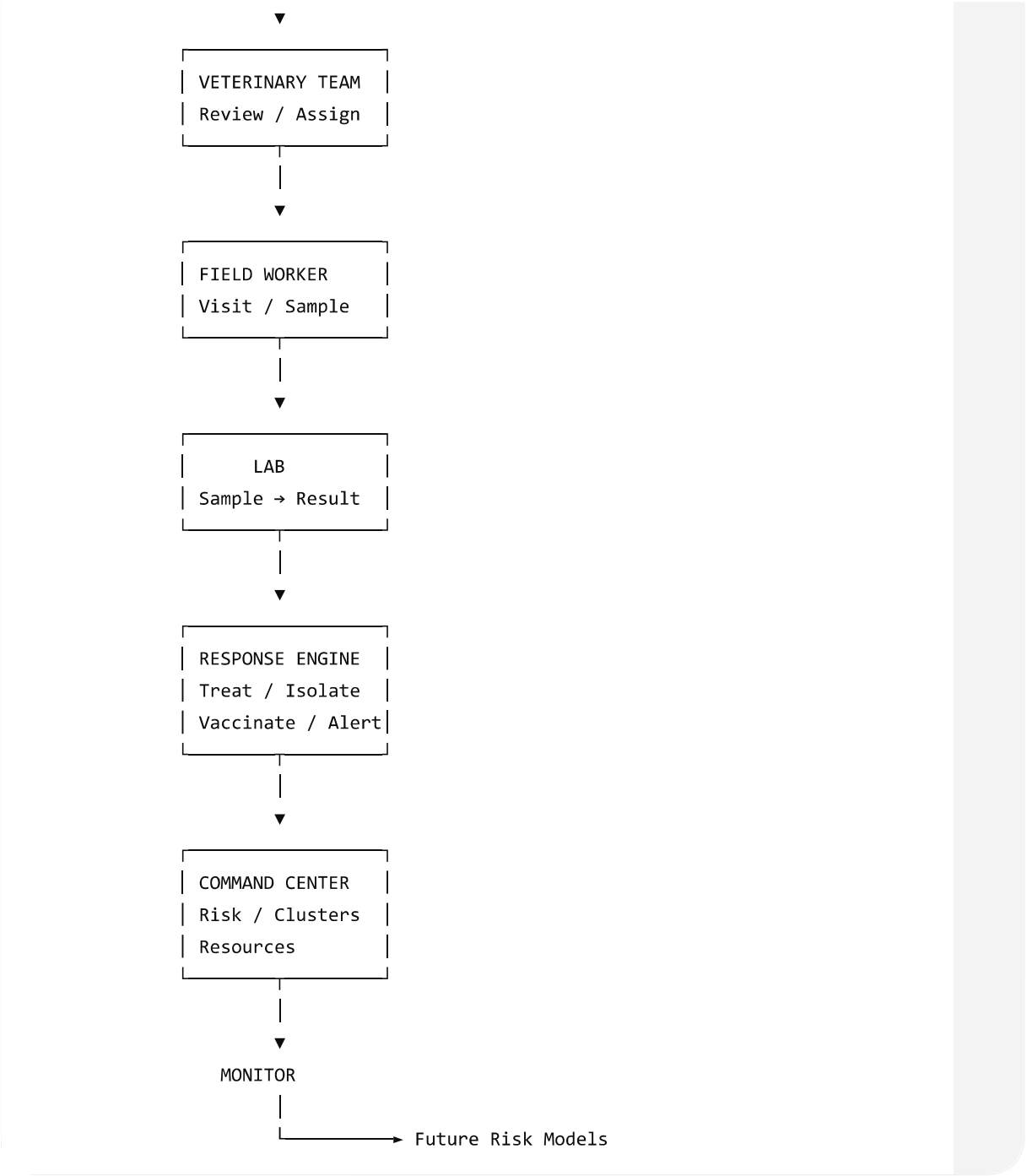
Case
↓
Cluster
↓
Risk map
↓
Government dashboard
↓
Vaccination priority
↓
Farmer advisory

This mirrors the PRD's recommended demo narrative.

35. Final Product Flow

The final architecture you should present is:





36. MVP vs Final Product

Capability	Hackathon MVP	Pilot	Full Product
Farmer reporting	Yes	Yes	Yes
Offline reporting	Yes	Yes	Yes

Capability	Hackathon MVP	Pilot	Full Product
AI triage	Rules + lightweight ML	Better ML	Advanced CDSS
GIS	Yes	Yes	Yes
Cluster detection	Basic spatial-temporal	SaTScan	Advanced
Weather forecasting	Seed/demo data	Yes	Yes
Herd record	Yes	Yes	Yes
Animal passport	No	Yes	Yes
Vet dashboard	Yes	Yes	Yes
Lab workflow	Yes	Yes	Yes
QR sample tracking	Yes	Yes	Yes
Vaccination	Yes	Yes	Yes
SMS alerts	Demo	Yes	Yes
IVR	No	Yes	Yes
Explainable AI	Basic	Yes	Yes
Multilingual	Basic	Yes	Advanced
IoT cold chain	No	Optional	Yes
National integration	No	No	Yes

The sequencing follows the PRD's Phase 1/2/3 roadmap.

37. The Most Important Design Principle

Your project should **not try to beat NADRES at forecasting**.

It should solve the missing connection:

Ground-level observation → AI triage → spatial detection → veterinary action → laboratory confirmation → prevention → government planning.

That is exactly where the research identifies the fragmentation problem.

So the strongest architecture is:

NADRES-style prediction + EMA-i-style reporting + AfyaData-style offline operation + SaTScan-style cluster detection + Bharat Pashudhan-style health records + LIMS-style laboratory traceability = PashuShield

But the important innovation is **not simply combining names/features**. It is creating a **single closed-loop workflow** in which information generated at one stage automatically drives the next stage.

38. Success Metrics

For the pilot, track the metrics already defined in the PRD:

KPI	Target
Report → triage	< 2 min
Suspected case → field assignment	< 4 hours
Offline reports successfully synced	>95%
Clusters detected before 3+ village self-report	≥60%
Pilot-district vaccination visibility	100%
Samples with visible end-to-end status	100%

Final recommendation